

```
In [3]: import pandas as pd
import matplotlib.pyplot as plt
import seaborn as sns
from sklearn.cluster import KMeans
from sklearn.preprocessing import MinMaxScaler
```

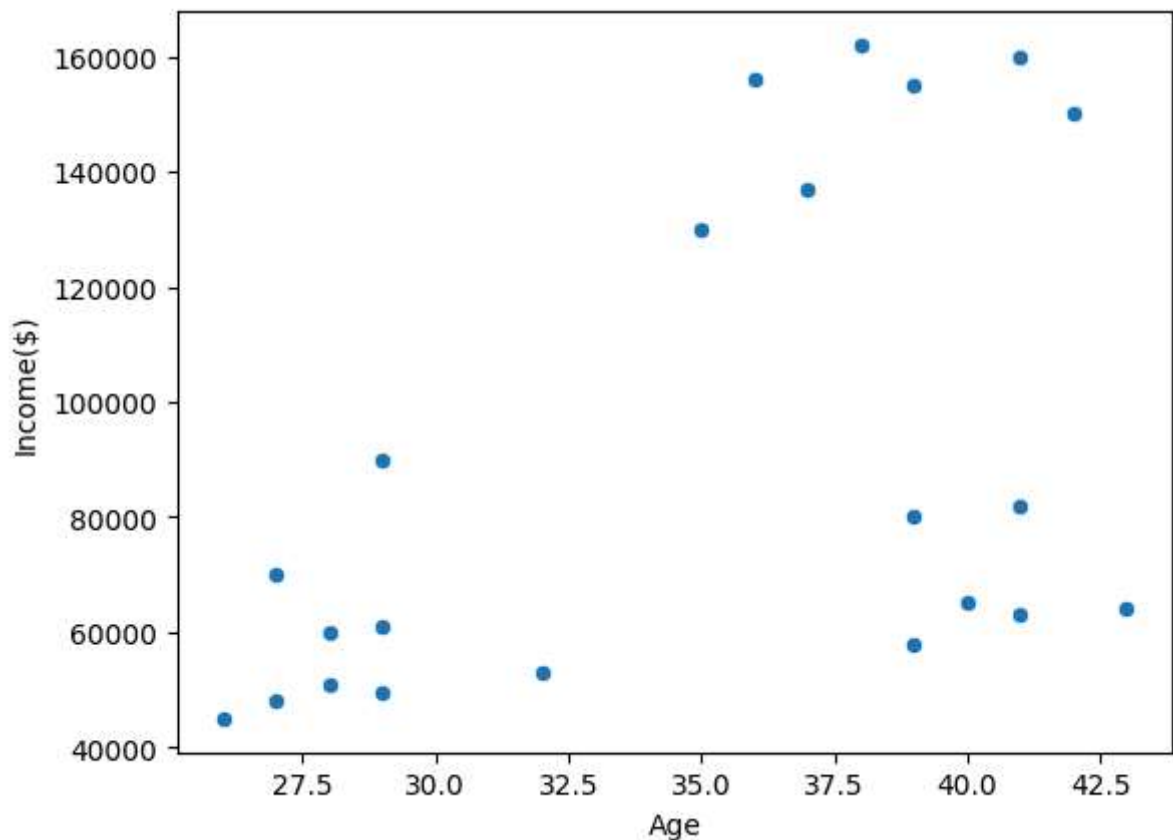
```
In [2]: df = pd.read_csv(r"income.csv")
df.head()
```

```
Out[2]:
```

	Name	Age	Income(\$)
0	Rob	27	70000
1	Michael	29	90000
2	Mohan	29	61000
3	Ismail	28	60000
4	Kory	42	150000

```
In [5]: sns.scatterplot(data=df, x='Age', y='Income($)')
```

```
Out[5]: <Axes: xlabel='Age', ylabel='Income($) '>
```



```
In [6]: km = KMeans(n_clusters=3)
km
```

Out[6]:

▼ KMeans ⓘ ?

► Parameters

```
In [7]: y_predict = km.fit_predict(df[['Age', 'Income($)']])
y_predict
```

```
Out[7]: array([1, 1, 1, 1, 0, 0, 0, 0, 0, 2, 2, 1, 1, 1, 1, 1, 1, 1, 1, 1],
      dtype=int32)
```

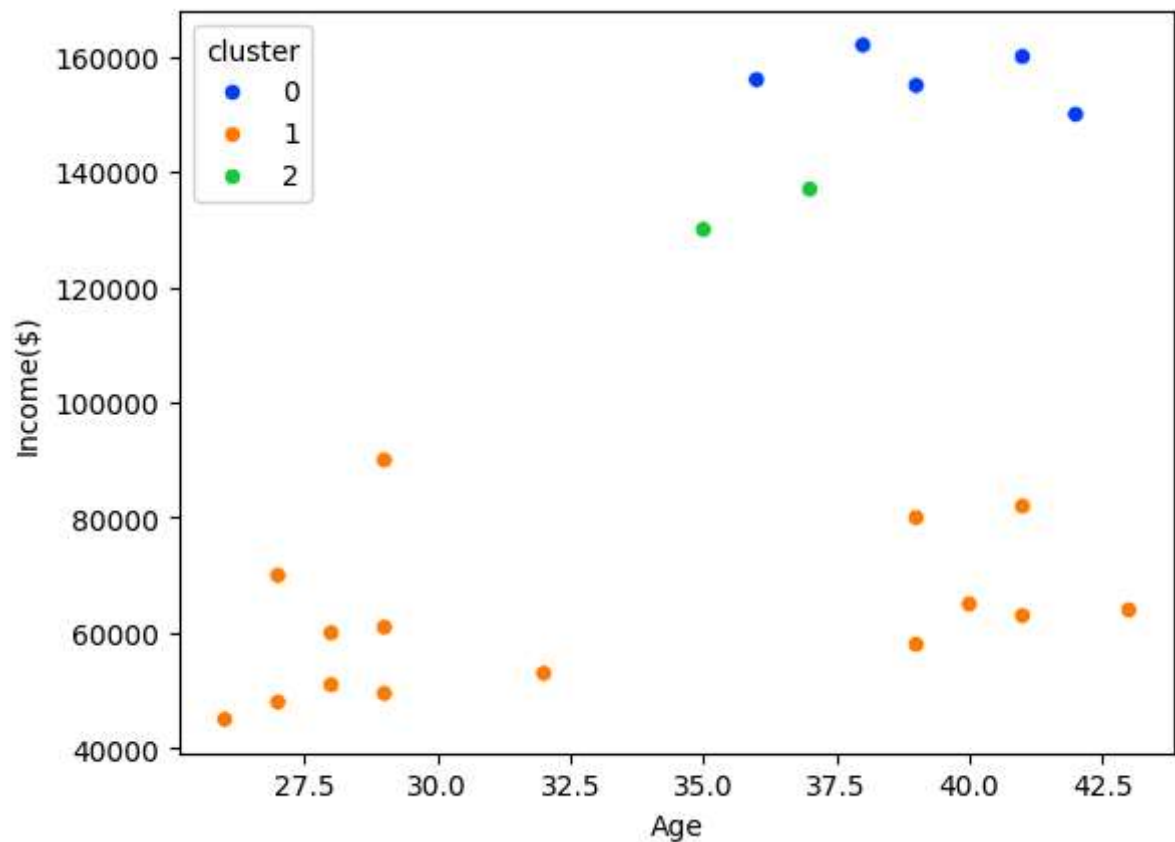
```
In [8]: df['cluster'] = y_predict
df.head()
```

```
Out[8]:
```

	Name	Age	Income(\$)	cluster
0	Rob	27	70000	1
1	Michael	29	90000	1
2	Mohan	29	61000	1
3	Ismail	28	60000	1
4	Kory	42	150000	0

```
In [9]: sns.scatterplot(data=df, x='Age', y='Income($)', hue='cluster', palette='bright')
```

```
Out[9]: <Axes: xlabel='Age', ylabel='Income($) '>
```



```
In [13]: # Scaling required for better clustering
# using Minmaxscaler
scaler = MinMaxScaler()
scaler.fit(df[['Income($)']])
df['Income($)'] = scaler.transform(df[['Income($)']])

scaler.fit(df[['Age']])
df['Age'] = scaler.transform(df[['Age']])

df
```

Out[13]:

	Name	Age	Income(\$)	cluster
0	Rob	0.058824	0.213675	1
1	Michael	0.176471	0.384615	1
2	Mohan	0.176471	0.136752	1
3	Ismail	0.117647	0.128205	1
4	Kory	0.941176	0.897436	0
5	Gautam	0.764706	0.940171	0
6	David	0.882353	0.982906	0
7	Andrea	0.705882	1.000000	0
8	Brad	0.588235	0.948718	0
9	Angelina	0.529412	0.726496	2
10	Donald	0.647059	0.786325	2
11	Tom	0.000000	0.000000	1
12	Arnold	0.058824	0.025641	1
13	Jared	0.117647	0.051282	1
14	Stark	0.176471	0.038462	1
15	Ranbir	0.352941	0.068376	1
16	Dipika	0.823529	0.170940	1
17	Priyanka	0.882353	0.153846	1
18	Nick	1.000000	0.162393	1
19	Alia	0.764706	0.299145	1
20	Sid	0.882353	0.316239	1
21	Abdul	0.764706	0.111111	1

```
In [15]: km = KMeans(n_clusters=3)
y_predict = km.fit_predict(df[['Age', 'Income($)']])
y_predict
```

```
Out[15]: array([0, 0, 0, 0, 1, 1, 1, 1, 1, 1, 1, 0, 0, 0, 0, 2, 2, 2, 2, 2, 2],
      dtype=int32)
```

```
In [19]: km.cluster_centers_
```

```
Out[19]: array([[0.1372549 , 0.11633428],
      [0.72268908, 0.8974359 ],
      [0.85294118, 0.2022792 ]])
```

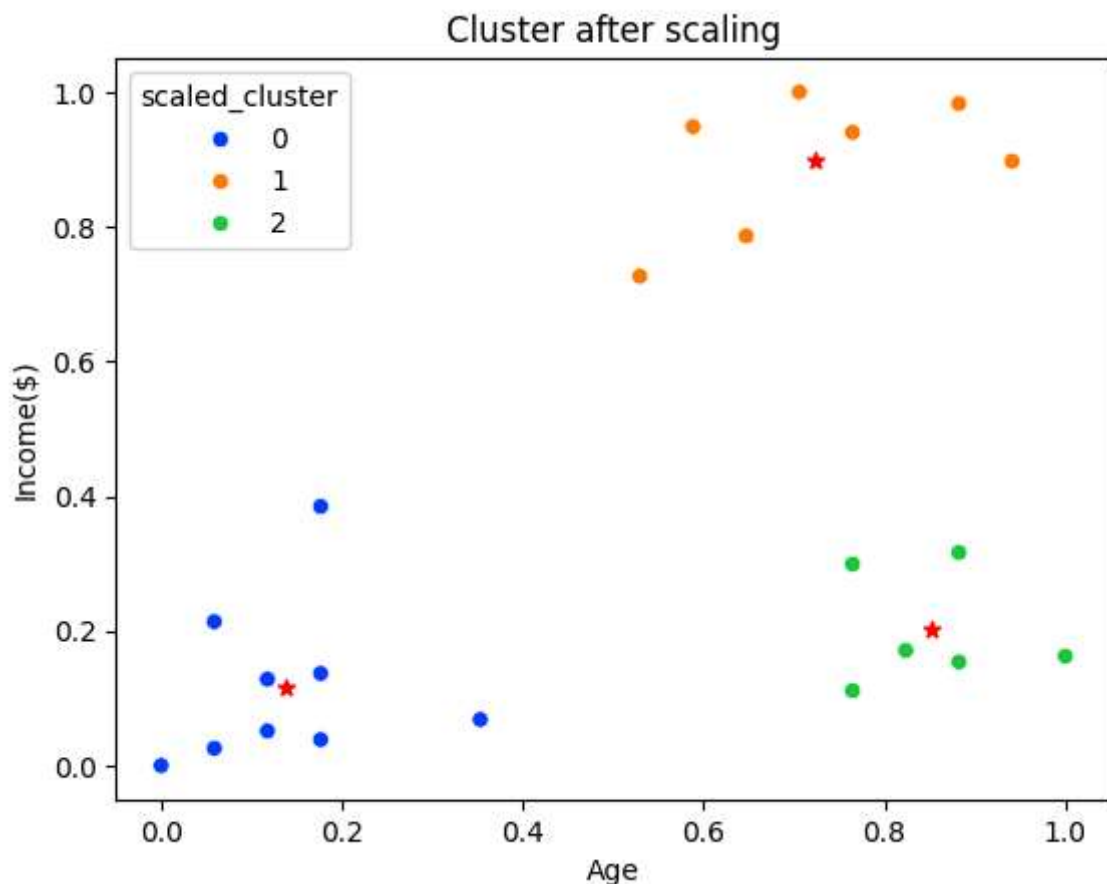
```
In [16]: df['scaled_cluster'] = y_predict
df.head()
```

```
Out[16]:
```

	Name	Age	Income(\$)	cluster	scaled_cluster
0	Rob	0.058824	0.213675	1	0
1	Michael	0.176471	0.384615	1	0
2	Mohan	0.176471	0.136752	1	0
3	Ismail	0.117647	0.128205	1	0
4	Kory	0.941176	0.897436	0	1

```
In [21]: sns.scatterplot(data=df, x='Age', y='Income($)', hue='scaled_cluster', palette='bri
plt.scatter(km.cluster_centers_[0], km.cluster_centers_[1], color='red', marker
plt.title("Cluster after scaling")
```

```
Out[21]: Text(0.5, 1.0, 'Cluster after scaling')
```



```
In [24]: # elbo method

k_rng = range(1,10)
sse = []

for k in k_rng:
    km = KMeans(n_clusters=k)
```

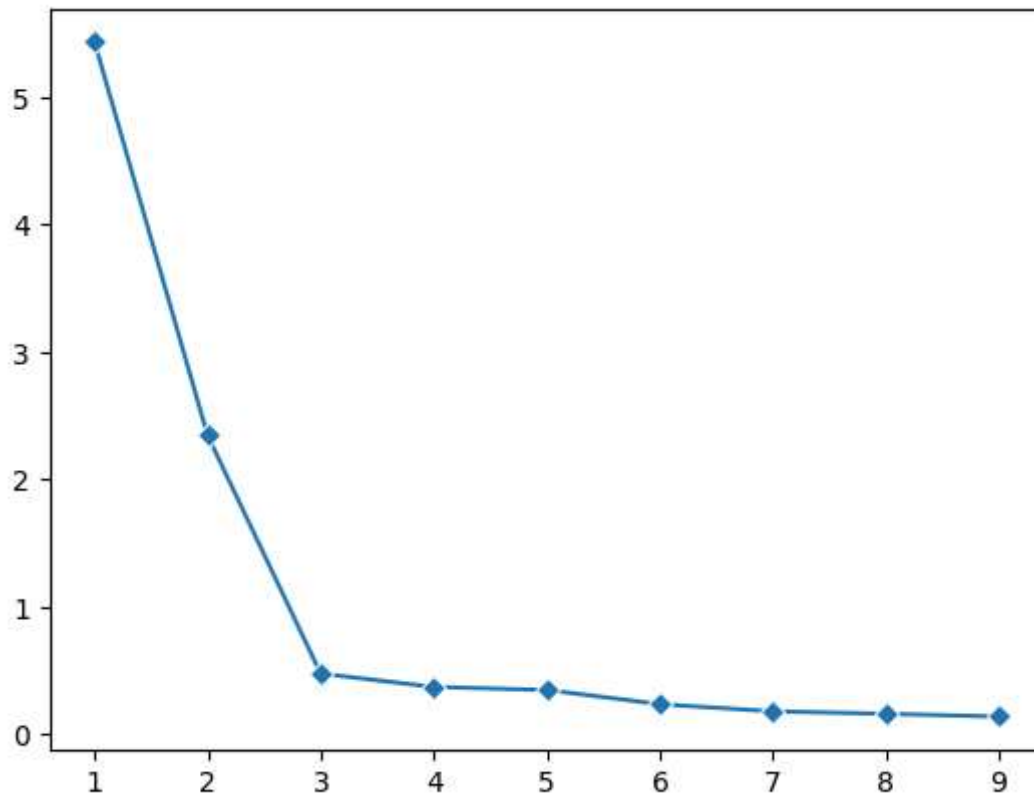
```
km.fit(df[['Age', 'Income($)']])
sse.append(km.inertia_) # returns sum of square error
```

In [25]: sse

Out[25]: [5.434011511988176,
2.3456144914725936,
0.47507834985530945,
0.3698658919234831,
0.34635005732105967,
0.23472396003560048,
0.17840674931327927,
0.1591933318857875,
0.13733705840476904]

In [28]: sns.lineplot(x = range(1,10), y = sse, marker='D')

Out[28]: <Axes: >



In [29]: # elbo position at '3'
n_cluster = 3

Iris Dataset

In [31]: # form the cluster of flowers using petal width and length features. Drop other fea
figure out if any scaling req
draw elbow plot

```
In [32]: data = sns.load_dataset('iris')
data.head()
```

```
Out[32]:
```

	sepal_length	sepal_width	petal_length	petal_width	species
0	5.1	3.5	1.4	0.2	setosa
1	4.9	3.0	1.4	0.2	setosa
2	4.7	3.2	1.3	0.2	setosa
3	4.6	3.1	1.5	0.2	setosa
4	5.0	3.6	1.4	0.2	setosa

```
In [34]: data = data.drop(['sepal_length', 'sepal_width'], axis=1)
```

```
In [35]: data.head()
```

```
Out[35]:
```

	petal_length	petal_width	species
0	1.4	0.2	setosa
1	1.4	0.2	setosa
2	1.3	0.2	setosa
3	1.5	0.2	setosa
4	1.4	0.2	setosa

```
In [36]: data.isna().sum()
```

```
Out[36]: petal_length    0
petal_width    0
species    0
dtype: int64
```

```
In [37]: data.shape
```

```
Out[37]: (150, 3)
```

```
In [38]: data.info()
```

```
<class 'pandas.DataFrame'>
RangeIndex: 150 entries, 0 to 149
Data columns (total 3 columns):
#   Column          Non-Null Count  Dtype
---  -
0   petal_length    150 non-null   float64
1   petal_width     150 non-null   float64
2   species         150 non-null   str
dtypes: float64(2), str(1)
memory usage: 4.9 KB
```

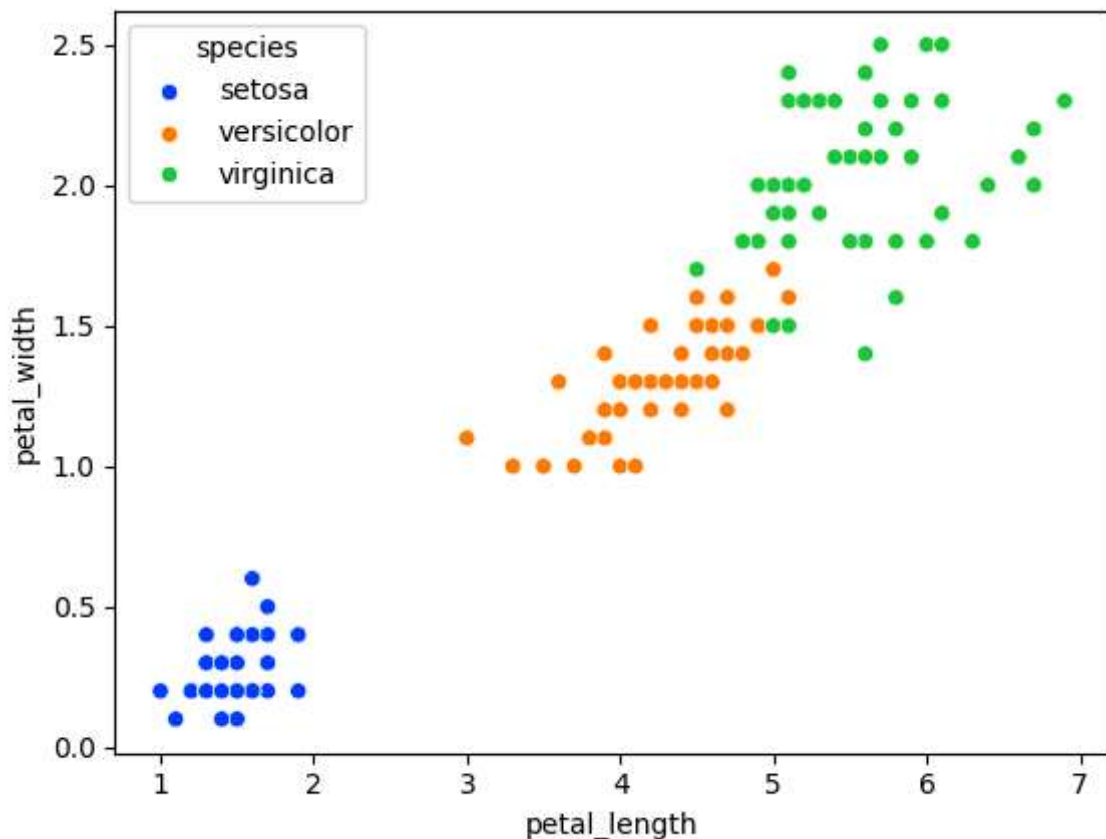
```
In [39]: data.describe()
```

```
Out[39]:
```

	petal_length	petal_width
count	150.000000	150.000000
mean	3.758000	1.199333
std	1.765298	0.762238
min	1.000000	0.100000
25%	1.600000	0.300000
50%	4.350000	1.300000
75%	5.100000	1.800000
max	6.900000	2.500000

```
In [41]: sns.scatterplot(data=data, x='petal_length', y='petal_width', hue='species', palette
```

```
Out[41]: <Axes: xlabel='petal_length', ylabel='petal_width'>
```



```
In [45]: # usign Elbo method to find cluster numbers
```

```
sse = [] # sum of square error
for k in range(2,10):
    km = KMeans(n_clusters=k)
    km.fit(data[['petal_length', 'petal_width']])
```

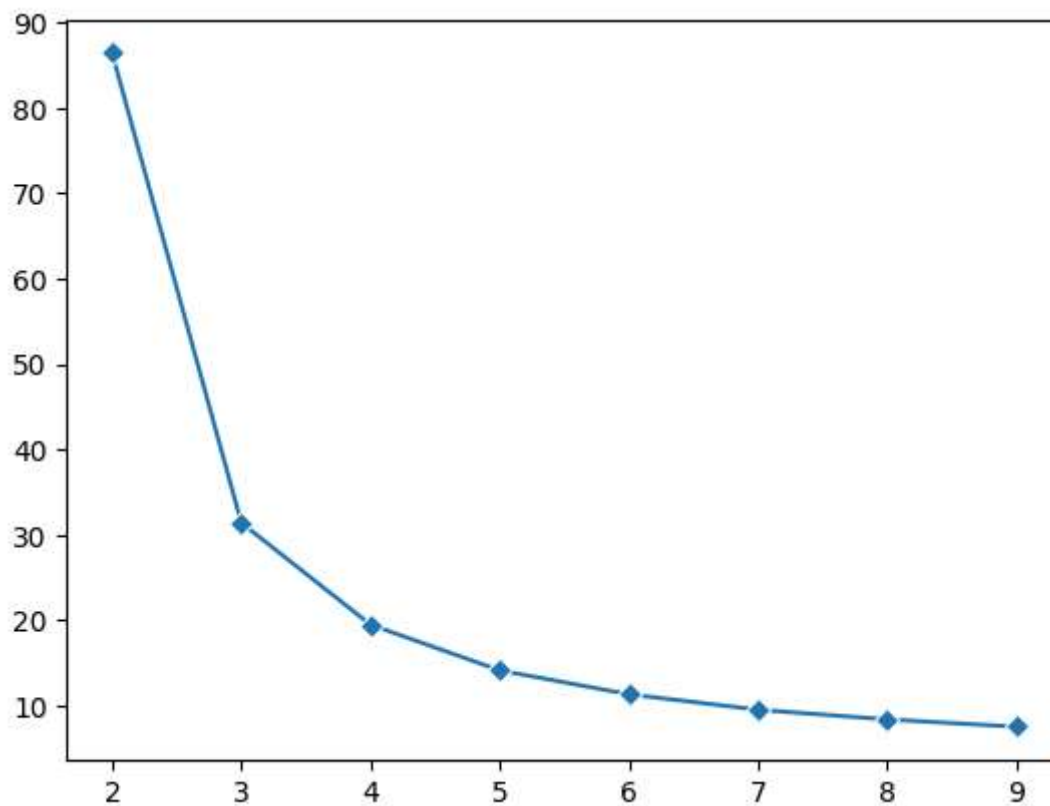
```
sse.append(km.inertia_)
```

In [46]: sse

```
Out[46]: [86.39021984551395,  
          31.412885668276978,  
          19.483000899685116,  
          14.130893864755937,  
          11.33106942325621,  
          9.517549993137,  
          8.387576355757133,  
          7.535215367707626]
```

In [48]: `sns.lineplot(x=range(2,10),y=sse, marker='D')`

Out[48]: <Axes: >



In [51]: `#lets take cluster = 3`

```
km = KMeans(n_clusters=3)  
y_predict = km.fit_predict(data[['petal_length', 'petal_width']])  
y_predict
```

```
Out[51]: array([0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0,
                0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0,
                0, 0, 0, 0, 0, 0, 2, 2, 2, 2, 2, 2, 2, 2, 2, 2, 2, 2, 2, 2, 2, 2,
                2, 2, 2, 2, 2, 2, 2, 2, 2, 2, 2, 2, 1, 2, 2, 2, 2, 2, 2, 1, 2, 2, 2, 2,
                2, 2, 2, 2, 2, 2, 2, 2, 2, 2, 2, 2, 2, 1, 1, 1, 1, 1, 1, 1, 2, 1, 1, 1,
                1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 2, 1, 1, 1, 1, 1, 1, 1, 2, 1, 1, 1, 1, 1,
                1, 1, 1, 1, 1, 1, 2, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1], dtype=int32)
```

```
In [55]: km.cluster_centers_
```

```
Out[55]: array([[1.462      , 0.246      ],
                [5.59583333, 2.0375      ],
                [4.26923077, 1.34230769]])
```

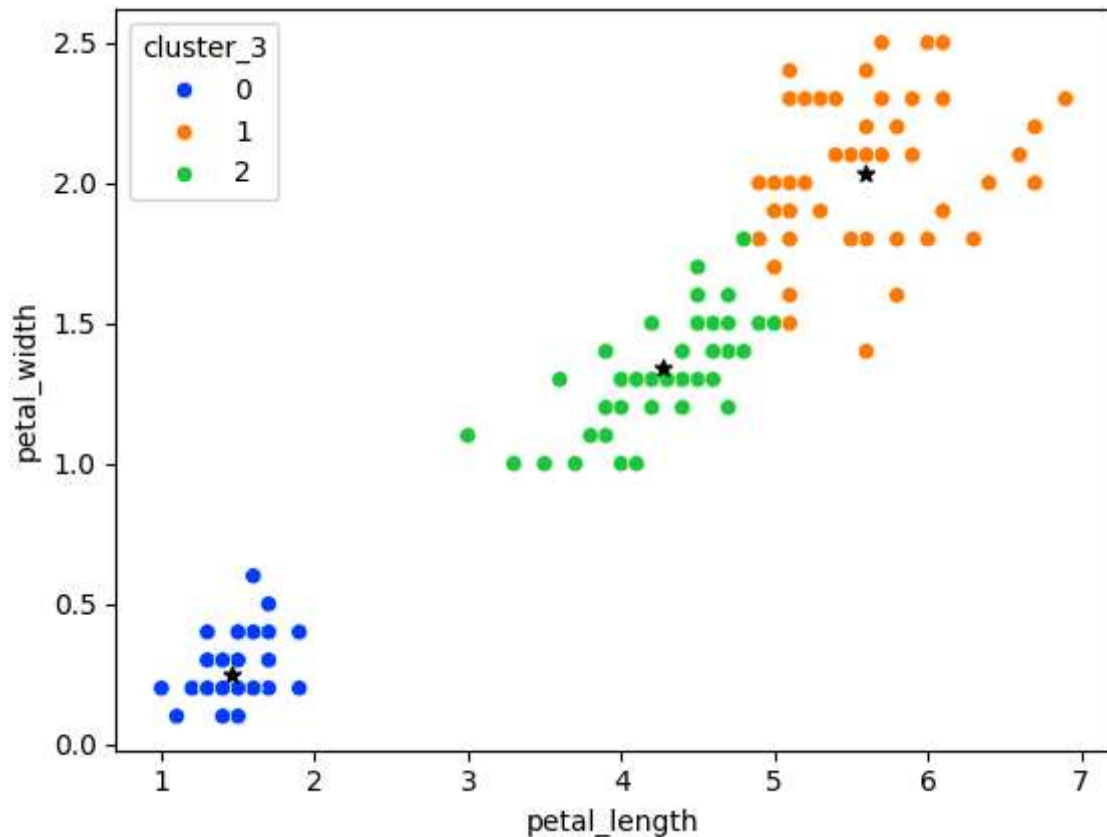
```
In [52]: data['cluster_3'] = y_predict
data.head()
```

```
Out[52]:
```

	petal_length	petal_width	species	cluster_3
0	1.4	0.2	setosa	0
1	1.4	0.2	setosa	0
2	1.3	0.2	setosa	0
3	1.5	0.2	setosa	0
4	1.4	0.2	setosa	0

```
In [58]: sns.scatterplot(data=data, x='petal_length', y='petal_width', hue='cluster_3', palette=
plt.scatter(km.cluster_centers_[0,0], km.cluster_centers_[0,1], color='black', mark
```

```
Out[58]: <matplotlib.collections.PathCollection at 0x274824f6d50>
```



In [61]: *# Lets do scaling the data using Minmaxscaler*

```
scaler = MinMaxScaler()
scaler.fit(data[['petal_length']])

data['petal_length'] = scaler.transform(data[['petal_length']])

scaler.fit(data[['petal_width']])
data['petal_width'] = scaler.transform(data[['petal_width']])
data.head()
```

Out[61]:

	petal_length	petal_width	species	cluster_3
0	0.067797	0.041667	setosa	0
1	0.067797	0.041667	setosa	0
2	0.050847	0.041667	setosa	0
3	0.084746	0.041667	setosa	0
4	0.067797	0.041667	setosa	0

In [62]:

```
km = KMeans(n_clusters=3)
y_predict = km.fit_predict(data[['petal_length','petal_width']])
y_predict
```

```
Out[62]: array([1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1,
                1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1,
                1, 1, 1, 1, 1, 1, 2, 2, 2, 2, 2, 2, 2, 2, 2, 2, 2, 2, 2, 2, 2, 2,
                2, 2, 2, 2, 0, 2, 2, 2, 2, 2, 2, 2, 0, 2, 2, 2, 2, 2, 2, 2, 2, 2,
                2, 2, 2, 2, 2, 2, 2, 2, 2, 2, 2, 2, 0, 0, 0, 0, 0, 0, 2, 0, 0, 0,
                0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 2, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0,
                0, 2, 2, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0], dtype=int32)
```

```
In [63]: data['scaled_cluster'] = y_predict
data.head()
```

```
Out[63]:
```

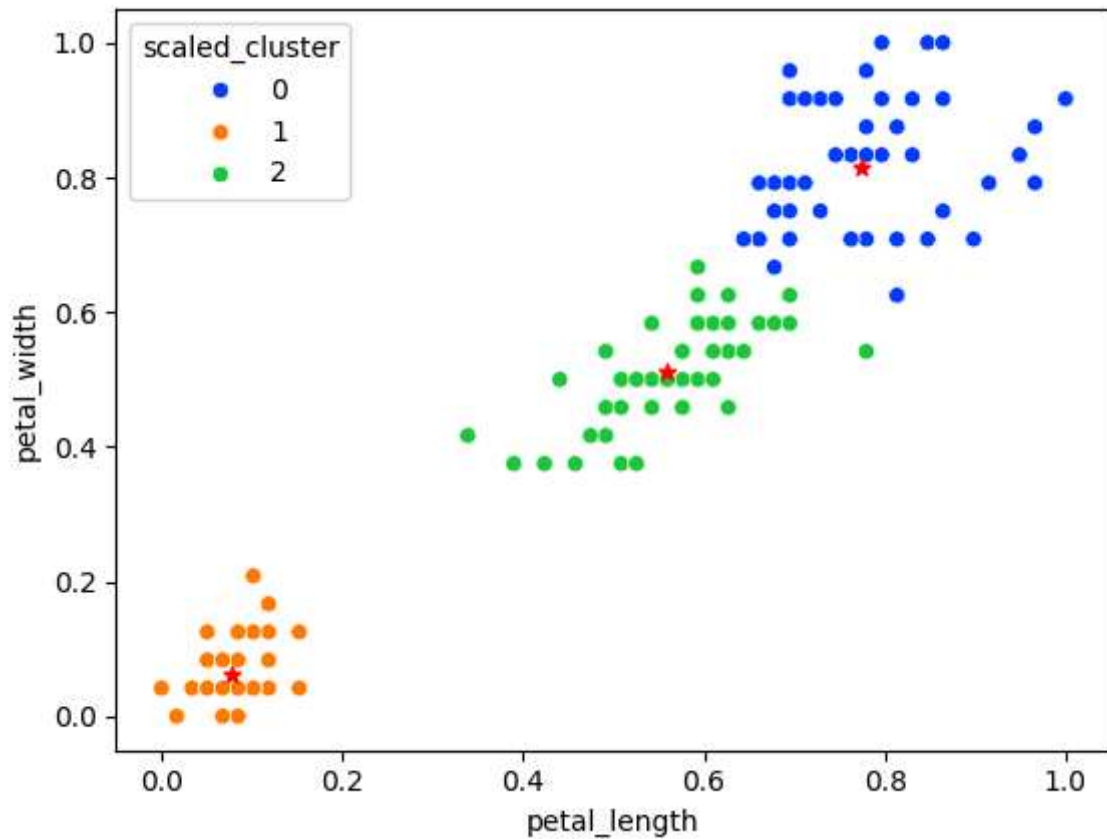
	petal_length	petal_width	species	cluster_3	scaled_cluster
0	0.067797	0.041667	setosa	0	1
1	0.067797	0.041667	setosa	0	1
2	0.050847	0.041667	setosa	0	1
3	0.084746	0.041667	setosa	0	1
4	0.067797	0.041667	setosa	0	1

```
In [65]: km.cluster_centers_
```

```
Out[65]: array([[0.7740113 , 0.81510417],
                [0.07830508, 0.06083333],
                [0.55867014, 0.51041667]])
```

```
In [68]: sns.scatterplot(data=data, x='petal_length', y='petal_width', hue='scaled_cluster',
                        plt.scatter(km.cluster_centers_[0], km.cluster_centers_[1], color='red', marker='x'))
```

```
Out[68]: <matplotlib.collections.PathCollection at 0x27482870cd0>
```



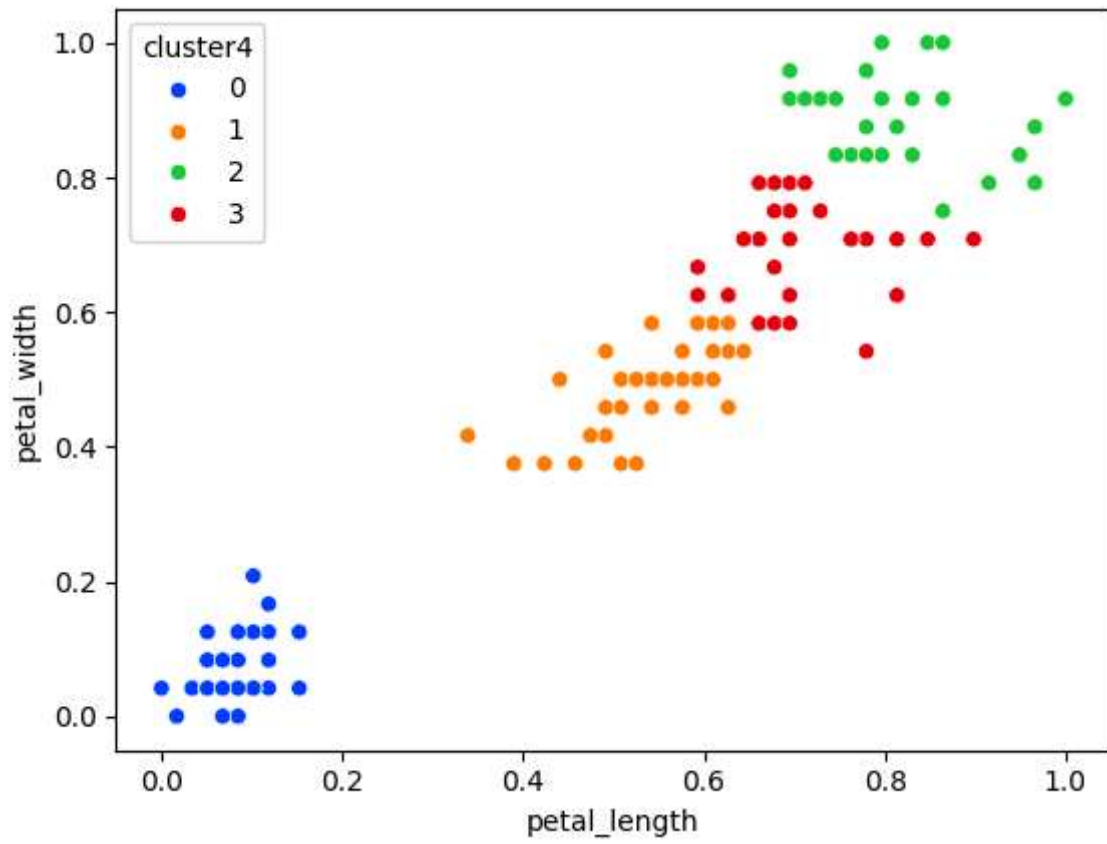
```
In [69]: # Lets take cluster = 4
km = KMeans(n_clusters=4)
y_pred = km.fit_predict(data[['petal_length', 'petal_width']])
data['cluster4'] = y_pred
data.head()
```

```
Out[69]:
```

	petal_length	petal_width	species	cluster_3	scaled_cluster	cluster4
0	0.067797	0.041667	setosa	0	1	0
1	0.067797	0.041667	setosa	0	1	0
2	0.050847	0.041667	setosa	0	1	0
3	0.084746	0.041667	setosa	0	1	0
4	0.067797	0.041667	setosa	0	1	0

```
In [71]: sns.scatterplot(data=data, x='petal_length', y='petal_width', hue='cluster4', palet
```

```
Out[71]: <Axes: xlabel='petal_length', ylabel='petal_width'>
```



In []: